

CAREER ADVANCEMENT SKILLS

Assignment-1

Name: Subhapreet Patro

Roll No.: 2211CS010547

Section: I

Q1) Write a java program to build a class Employee having name, department, salary & performanceRating as attributes. And using Java Streams :

Filters the employees who have a performance rating of 4.0 or higher.

Find the highest paid employee after filtering.

Print the salary of the highest-paid employee with rating above 4.0

A)

Program:

```
import java.util.*;
```

```
import java.util.stream.*;
```

```
class Employee {
```

```
    private String name;
```

```
    private String department;
```

```
    private double salary;
```

```
    private double performanceRating;
```

```
    public Employee(String name, String department, double salary, double  
performanceRating) {
```

```
        this.name = name;
```

```
        this.department = department;
```

```
        this.salary = salary;
```

```
        this.performanceRating = performanceRating;
```

```
    }
```

```
    public double getSalary() {
```

```
        return salary;
    }

    public double getPerformanceRating() {
        return performanceRating;
    }

    public String getName() {
        return name;
    }

    // To print the Employee object nicely
    @Override
    public String toString() {
        return name + " (" + department + ") - Salary: " + salary + ", Rating: " +
performanceRating;
    }
}

public class Main {
    public static void main(String[] args) {
        // Sample Employee list
        List<Employee> employees = Arrays.asList(
            new Employee("Alice", "IT", 75000, 4.5),
            new Employee("Bob", "HR", 60000, 3.9),
            new Employee("Charlie", "Sales", 82000, 4.2),
            new Employee("Diana", "Marketing", 91000, 4.7),
            new Employee("Eve", "IT", 70000, 4.0)
        );

        // Use Stream to filter and find the highest-paid employee with rating >= 4.0
```

```

Optional<Employee> highestPaid = employees.stream()
    .filter(e -> e.getPerformanceRating() >= 4.0)
    .max(Comparator.comparingDouble(Employee::getSalary));

if (highestPaid.isPresent()) {
    System.out.println("Highest paid employee with rating >= 4.0:");
    System.out.println(highestPaid.get());
    System.out.println("Salary: " + highestPaid.get().getSalary());
} else {
    System.out.println("No employee found with rating >= 4.0");
}
}
}

```

Output:

Highest paid employee with rating >= 4.0:
Diana (Marketing) - Salary: 91000.0, Rating: 4.7
Salary: 91000.0

Q2) Create a student class with roll number, name & address as data members, initialize and display student details.

Student Class
Rollno name address
addStudentRecord() displayDetails()

Test Class		
S1	S2	S3
101 Akhil HYD	102 Bindhu MRUH	103 Chanran SEC
main()		

A)

Program:

```
class Student {
```

```
int rollNo;  
String name;  
String address;
```

```
// Method to add student details
```

```
public void addStudentRecord(int rollNo, String name, String address) {  
    this.rollNo = rollNo;  
    this.name = name;  
    this.address = address;  
}
```

```
// Method to display student details
```

```
public void displayDetails() {  
    System.out.println("Roll No: " + rollNo);  
    System.out.println("Name: " + name);  
    System.out.println("Address: " + address);  
    System.out.println("-----");  
}  
}
```

```
public class TestStudent {  
    public static void main(String[] args) {  
        // Creating three student objects  
        Student s1 = new Student();  
        Student s2 = new Student();  
        Student s3 = new Student();  
  
        // Adding records  
        s1.addStudentRecord(101, "Akhil", "HYD");  
        s2.addStudentRecord(102, "Bindhu", "MRUH");  
        s3.addStudentRecord(103, "Chanran", "SEC");  
    }  
}
```

```
// Displaying details  
System.out.println("Student Details:");  
  
s1.displayDetails();  
s2.displayDetails();  
s3.displayDetails();  
  
}  
}
```

Output:

Student Details:

Roll No: 101

Name: Akhil

Address: HYD

Roll No: 102

Name: Bindhu

Address: MRUH

Roll No: 103

Name: Chanran

Address: SEC

Q3) How to copy an object in Java?

A)

In Java, **copying an object** means creating a new object with the **same data as an existing one**. There are several ways to do this:

1. Using a Copy Constructor

A **copy constructor** is a constructor that takes another object of the same class as an argument and copies its fields.

Example:

```
class Student {
```

```
int rollNo;

String name;

// Constructor
Student(int rollNo, String name) {
    this.rollNo = rollNo;
    this.name = name;
}

// Copy constructor
Student(Student s) {
    this.rollNo = s.rollNo;
    this.name = s.name;
}

void display() {
    System.out.println("Roll No: " + rollNo + ", Name: " + name);
}

}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(101, "Akhil");
        Student s2 = new Student(s1); // Copying s1 into s2

        System.out.println("Original Student:");
        s1.display();

        System.out.println("Copied Student:");
        s2.display();
    }
}
```

```
}
```

Output:

Original Student:

Roll No: 101, Name: Akhil

Copied Student:

Roll No: 101, Name: Akhil

2. Using the clone() Method

- To use clone(), a class must implement the Cloneable interface and override the clone() method.
- This performs a **shallow copy**.

Example:

```
class Student implements Cloneable {  
    int rollNo;  
    String name;  
  
    Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
  
    public Object clone() throws CloneNotSupportedException {  
        return super.clone(); // Shallow copy  
    }  
  
    void display() {  
        System.out.println("Roll No: " + rollNo + ", Name: " + name);  
    }  
}  
  
public class Main {
```

```
public static void main(String[] args) {  
    try {  
        Student s1 = new Student(102, "Bindhu");  
        Student s2 = (Student) s1.clone(); // Cloning s1  
  
        System.out.println("Original Student:");  
        s1.display();  
  
        System.out.println("Cloned Student:");  
        s2.display();  
    } catch (CloneNotSupportedException e) {  
        System.out.println("Cloning not supported.");  
    }  
}
```

Output:

Original Student:

Roll No: 102, Name: Bindhu

Cloned Student:

Roll No: 102, Name: Bindhu

3. Using Serialization and Deserialization (Deep Copy)

- A **deep copy** duplicates not only the object but also **all the objects it refers to**, recursively.
- Unlike shallow copy, changes in the copied object **do not affect** the original one.

Deep Copy Using Serialization

To do this:

1. Your class must implement the Serializable interface.
 2. Use ObjectOutputStream to write the object to a byte stream.
 3. Use ObjectInputStream to read it back and create a deep copy.
-

Example Program

```
import java.io.*;

// Student class implementing Serializable
class Student implements Serializable {
    int rollNo;
    String name;
    Address address;

    Student(int rollNo, String name, Address address) {
        this.rollNo = rollNo;
        this.name = name;
        this.address = address;
    }

    void display() {
        System.out.println("Roll No: " + rollNo + ", Name: " + name);
        System.out.println("Address: " + address.city + ", " + address.state);
    }
}

// Address class also needs to be Serializable
class Address implements Serializable {
    String city;
    String state;
    Address(String city, String state) {
        this.city = city;
        this.state = state;
    }
}

public class DeepCopyExample {
```

```

// Method to perform deep copy using serialization
public static <T> T deepCopy(T object) throws IOException, ClassNotFoundException {
    ByteArrayOutputStream byteOut = new ByteArrayOutputStream();
    ObjectOutputStream out = new ObjectOutputStream(byteOut);
    out.writeObject(object);

    ByteArrayInputStream byteIn = new ByteArrayInputStream(byteOut.toByteArray());
    ObjectInputStream in = new ObjectInputStream(byteIn);
    return (T) in.readObject();
}

public static void main(String[] args) {
    try {
        // Original object
        Address address = new Address("HYD", "Telangana");
        Student original = new Student(101, "Akhil", address);

        // Deep copy of the original object
        Student copied = deepCopy(original);

        // Modify the copied object's address
        copied.address.city = "SEC";
        copied.name = "Changed Name";

        System.out.println("Original Student:");
        original.display();

        System.out.println("\nCopied Student (after modification):");
        copied.display();

    } catch (Exception e) {

```

```

        e.printStackTrace();
    }
}
}

```

Output:

Original Student:

Roll No: 101, Name: Akhil

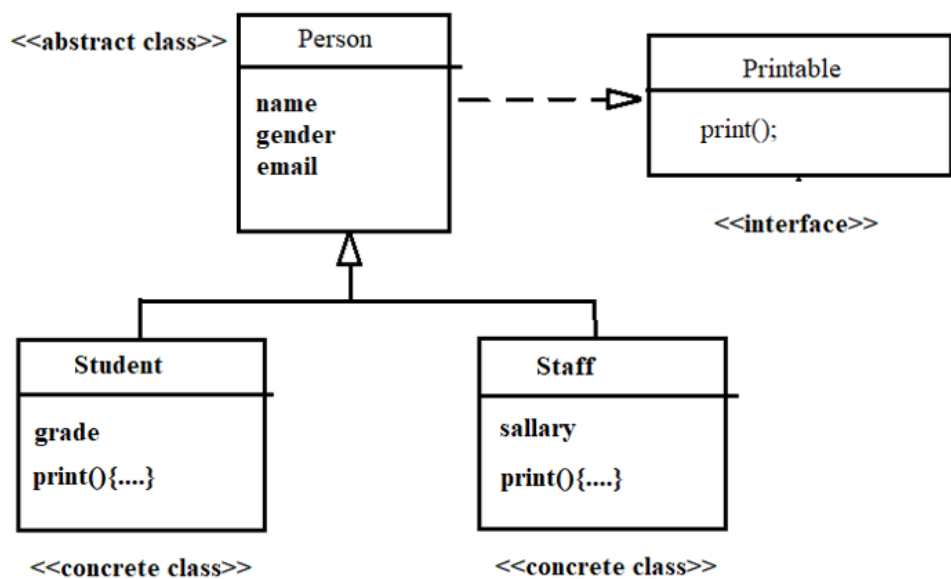
Address: HYD, Telangana

Copied Student (after modification):

Roll No: 101, Name: Changed Name

Address: SEC, Telangana

Q4) Build java program for the following class structure.



A)

Program:

```

// Printable interface
interface Printable {
    void print();
}

```

```
// Abstract class Person
abstract class Person implements Printable {
    String name;
    String gender;
    String email;

    // Constructor
    public Person(String name, String gender, String email) {
        this.name = name;
        this.gender = gender;
        this.email = email;
    }
}

// Concrete class Student
class Student extends Person {
    String grade;

    public Student(String name, String gender, String email, String grade) {
        super(name, gender, email);
        this.grade = grade;
    }

    @Override
    public void print() {
        System.out.println("Student Details:");
        System.out.println("Name : " + name);
        System.out.println("Gender : " + gender);
        System.out.println("Email : " + email);
        System.out.println("Grade : " + grade);
        System.out.println("-----");
    }
}
```

```
}  
}
```

```
// Concrete class Staff
```

```
class Staff extends Person {
```

```
    double salary;
```

```
    public Staff(String name, String gender, String email, double salary) {
```

```
        super(name, gender, email);
```

```
        this.salary = salary;
```

```
    }
```

```
@Override
```

```
public void print() {
```

```
    System.out.println("Staff Details:");
```

```
    System.out.println("Name   : " + name);
```

```
    System.out.println("Gender : " + gender);
```

```
    System.out.println("Email  : " + email);
```

```
    System.out.println("Salary : " + salary);
```

```
    System.out.println("-----");
```

```
}
```

```
}
```

```
// Test class
```

```
public class TestPerson {
```

```
    public static void main(String[] args) {
```

```
        Student student = new Student("Akhil", "Male", "akhil@example.com", "A+");
```

```
        Staff staff = new Staff("Bindhu", "Female", "bindhu@example.com", 50000.00);
```

```
// Polymorphism via interface
```

```
Printable p1 = student;
```

```

Printable p2 = staff;

p1.print();
p2.print();
}
}

```

Output:

Student Details:

Name : Akhil

Gender : Male

Email : akhil@example.com

Grade : A+

Staff Details:

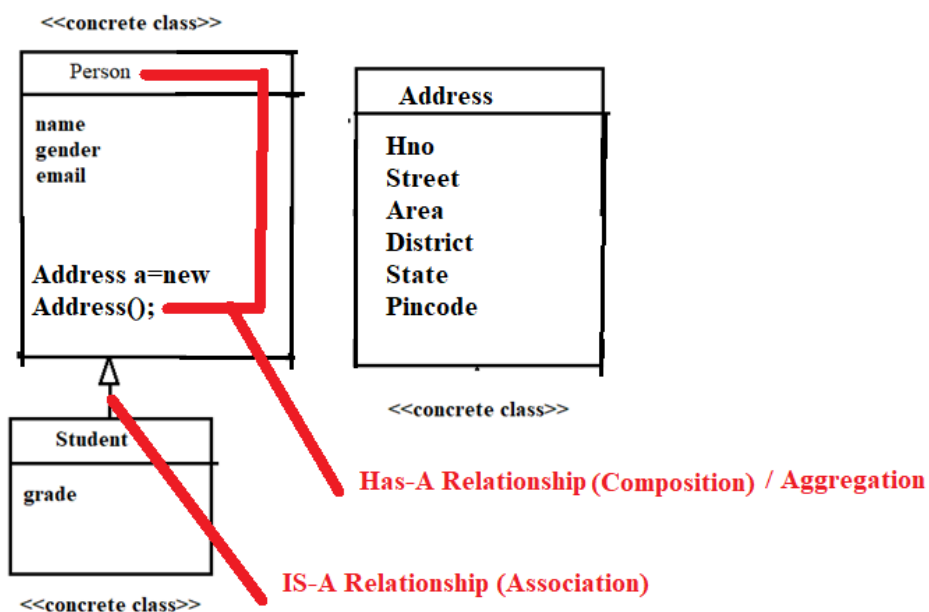
Name : Bindhu

Gender : Female

Email : bindhu@example.com

Salary : 50000.0

Q5) Build java program for the following class structure.



A)

Program:

```
// Address class
class Address {
    String hno;
    String street;
    String area;
    String district;
    String state;
    String pincode;

    public Address(String hno, String street, String area, String district, String state, String pincode) {
        this.hno = hno;
        this.street = street;
        this.area = area;
        this.district = district;
        this.state = state;
        this.pincode = pincode;
    }

    public void displayAddress() {
        System.out.println("Address:");
        System.out.println("House No: " + hno);
        System.out.println("Street : " + street);
        System.out.println("Area : " + area);
        System.out.println("District: " + district);
        System.out.println("State : " + state);
        System.out.println("Pincode : " + pincode);
    }
}
```

```
// Person class (Has-A Address)
class Person {
    String name;
    String gender;
    String email;
    Address address; // Has-A relationship

    public Person(String name, String gender, String email, Address address) {
        this.name = name;
        this.gender = gender;
        this.email = email;
        this.address = address;
    }

    public void displayPerson() {
        System.out.println("Name : " + name);
        System.out.println("Gender: " + gender);
        System.out.println("Email : " + email);
        address.displayAddress();
    }
}

// Student class (IS-A Person)
class Student extends Person {
    String grade;

    public Student(String name, String gender, String email, Address address, String grade) {
        super(name, gender, email, address);
        this.grade = grade;
    }
}
```



```
public void displayStudent() {  
    displayPerson(); // Call parent method  
    System.out.println("Grade : " + grade);  
}  
}  
  
// Test class  
public class TestStructure {  
    public static void main(String[] args) {  
        // Create Address object  
        Address addr = new Address("10-2", "MG Road", "Begumpet", "Hyderabad",  
"Telangana", "500016");  
        // Create Student object with Address  
        Student student = new Student("Akhil", "Male", "akhil@example.com", addr, "A+");  
  
        // Display Student details  
        System.out.println("Student Information:");  
        student.displayStudent();  
    }  
}
```

Output:

Student Information:

Name : Akhil

Gender: Male

Email : akhil@example.com

Address:

House No: 10-2

Street : MG Road

Area : Begumpet

District: Hyderabad

State : Telangana

Pincode : 500016

Grade : A+

Q6) Create a BankAccount class with an Inner class Interest implementing a scenario such that only an authenticated bank person can interact with inner class by calling contact() method of BankAccount. When contact() method will be called, it will verify with password and then he will be able to use the inner class and access it.

A)

Program:

```
import java.util.Scanner;
```

```
public class BankAccount {
    private String accountHolder;
    private double balance;
    private final String password = "admin123"; // predefined password
    public BankAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    // Inner class
    private class Interest {
        double rate = 5.0; // 5% interest

        public void calculateInterest() {
            double interest = (balance * rate) / 100;
            System.out.println("Interest on the current balance: ₹" + interest);
        }
    }

    // Method to authenticate and allow access to the Interest inner class
    public void contact() {
```

```

Scanner scanner = new Scanner(System.in);
System.out.print("Enter authentication password to access Interest details: ");
String inputPassword = scanner.nextLine();

if (inputPassword.equals(password)) {
    System.out.println("Authentication successful. Access granted.");
    Interest interest = new Interest();
    interest.calculateInterest();
} else {
    System.out.println("Authentication failed. Access denied.");
}
}

// Display basic account info
public void displayAccountInfo() {
    System.out.println("Account Holder: " + accountHolder);
    System.out.println("Balance: ₹" + balance);
}

// Main method to test the scenario
public static void main(String[] args) {
    BankAccount account = new BankAccount("Akhil", 100000.0);
    account.displayAccountInfo();
    account.contact();
}
}

```

Output:

Account Holder: Akhil

Balance: ₹100000.0

Enter authentication password to access Interest details: admin123

Authentication successful. Access granted.

Interest on the current balance: ₹5000.0

Q7) A software development company is designing a library management system that requires a flexible data container to store different types of items, such as books, magazines, and DVDs. The company wants a generic class that can store any type of object

A)

We'll define a **generic class LibraryItem<T>**, where T can be any object type. This provides flexibility and **type safety**.

Step-by-Step Java Program

1. Define sample item classes (Book, Magazine, DVD)

```
class Book {  
    String title;  
    String author;  
  
    Book(String title, String author) {  
        this.title = title;  
        this.author = author;  
    }  
  
    public void display() {  
        System.out.println("Book: " + title + " by " + author);  
    }  
}
```

```
class Magazine {  
    String name;  
    int issue;  
  
    Magazine(String name, int issue) {  
        this.name = name;  
        this.issue = issue;  
    }  
  
    public void display() {
```

```
        System.out.println("Magazine: " + name + ", Issue: " + issue);
    }
}

class DVD {
    String title;
    int duration; // in minutes

    DVD(String title, int duration) {
        this.title = title;
        this.duration = duration;
    }

    public void display() {
        System.out.println("DVD: " + title + ", Duration: " + duration + " mins");
    }
}
```

2. Create the Generic Class

```
// Generic class
class LibraryItem<T> {
    private T item;

    public LibraryItem(T item) {
        this.item = item;
    }

    public T getItem() {
        return item;
    }
}
```

```

public void displayItem() {
    if (item instanceof Book) {
        ((Book) item).display();
    } else if (item instanceof Magazine) {
        ((Magazine) item).display();
    } else if (item instanceof DVD) {
        ((DVD) item).display();
    } else {
        System.out.println("Unknown item type.");
    }
}
}

```

3. Test the Implementation

```

public class LibraryTest {
    public static void main(String[] args) {
        // Create items
        Book book = new Book("Clean Code", "Robert C. Martin");
        Magazine magazine = new Magazine("National Geographic", 202);
        DVD dvd = new DVD("Inception", 148);

        // Store items using generic class
        LibraryItem<Book> bookItem = new LibraryItem<>(book);
        LibraryItem<Magazine> magazineItem = new LibraryItem<>(magazine);
        LibraryItem<DVD> dvdItem = new LibraryItem<>(dvd);

        // Display stored items
        bookItem.displayItem();
        magazineItem.displayItem();
        dvdItem.displayItem();
    }
}

```

```
}
```

Output:

Book: Clean Code by Robert C. Martin

Magazine: National Geographic, Issue: 202

DVD: Inception, Duration: 148 mins

Q8) Java Regular Expression Example: Email Validation**A)****Program:**

```
import java.util.Scanner;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class EmailValidator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Simple regex pattern for email validation
        String emailRegex = "^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";

        // Compile the regex
        Pattern pattern = Pattern.compile(emailRegex);

        System.out.print("Enter an email address to validate: ");
        String email = scanner.nextLine();

        // Match the input with the regex
        Matcher matcher = pattern.matcher(email);

        if (matcher.matches()) {
            System.out.println("Valid email address.");
        }
    }
}
```

```
    } else {  
        System.out.println("Invalid email address.");  
    }  
}  
}
```

Output:

Enter an email address to validate: 2211cs010547@mallareddyuniversity.ac.in

Valid email address.

Q9) For the Application on International flight booking system, the system needs to calculate the arrival time of a flight based on:

- 1. Departure time (Local Time Zone)**
- 2. Flight duration**
- 3. Destination time zone**

Your task is to write a Java program that:

- Takes a departure time in one time zone.**
- Adds the flight duration to determine the arrival time.**

Converts the arrival time to the destination's time zone.

A)

Program:

```
import java.time.*;  
import java.time.format.DateTimeFormatter;  
import java.util.Scanner;  
  
public class FlightBookingSystem {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        // Formatter for date-time output  
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm z");
```



```
// 1. Input: Departure Time and Time Zone
System.out.print("Enter departure date and time (yyyy-MM-dd HH:mm): ");
String departureInput = scanner.nextLine();

System.out.print("Enter departure time zone (e.g., Asia/Kolkata): ");
String departureZoneStr = scanner.nextLine();

// 2. Input: Flight Duration
System.out.print("Enter flight duration in hours: ");
int hours = scanner.nextInt();

System.out.print("Enter flight duration in minutes: ");
int minutes = scanner.nextInt();
scanner.nextLine(); // consume newline

// 3. Input: Destination Time Zone
System.out.print("Enter destination time zone (e.g., Europe/London): ");
String destinationZoneStr = scanner.nextLine();

// 4. Parse and calculate
try {
    LocalDateTime departureDateTime = LocalDateTime.parse(departureInput,
DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"));
    ZoneId departureZone = ZoneId.of(departureZoneStr);
    ZoneId destinationZone = ZoneId.of(destinationZoneStr);

    // Create ZonedDateTime for departure
    ZonedDateTime zonedDeparture = ZonedDateTime.of(departureDateTime,
departureZone);

    // Add flight duration
```

```

        Duration flightDuration = Duration.ofHours(hours).plusMinutes(minutes);
        ZonedDateTime zonedArrival = zonedDeparture.plus(flightDuration);

        // Convert arrival time to destination time zone
        ZonedDateTime destinationArrival =
zonedArrival.withZoneSameInstant(destinationZone);

        // Output results
        System.out.println("\nFlight Details:");
        System.out.println("Departure: " + zonedDeparture.format(formatter));
        System.out.println("Arrival:  " + destinationArrival.format(formatter));
    } catch (Exception e) {
        System.out.println("Invalid input. Please check date, time, or time zone format.");
    }
}
}

```

Output:

Enter departure date and time (yyyy-MM-dd HH:mm): 2025-07-17 15:30

Enter departure time zone (e.g., Asia/Kolkata): Asia/Kolkata

Enter flight duration in hours: 10

Enter flight duration in minutes: 30

Enter destination time zone (e.g., Europe/London): Europe/London

Flight Details:

Departure: 2025-07-17 15:30 IST

Arrival: 2025-07-17 21:30 BST

Q10) Bank Account Withdrawal System with Custom Exceptions

Design a Java application to simulate a basic bank withdrawal system. The system must include robust error handling using user-defined exceptions to enforce business rules and ensure valid transactions.

A)

Program:

```
import java.util.Scanner;

// Custom exception for insufficient balance
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Custom exception for invalid (negative) amount
class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}

class BankAccount {
    private String accountHolder;
    private double balance;

    public BankAccount(String accountHolder, double initialBalance) {
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    public void deposit(double amount) throws InvalidAmountException {
        if (amount <= 0)
            throw new InvalidAmountException("Deposit amount must be greater than zero.");
        balance += amount;
        System.out.println("Deposited: ₹" + amount);
    }
}
```

```
public void withdraw(double amount) throws InsufficientBalanceException,
InvalidAmountException {
    if (amount <= 0)
        throw new InvalidAmountException("Withdrawal amount must be greater than
zero.");

    if (balance == 0)
        throw new InsufficientBalanceException("Withdrawal failed. Account has zero
balance.");

    if (amount > balance)
        throw new InsufficientBalanceException("Withdrawal failed. Insufficient balance.
Current balance: ₹" + balance);

    balance -= amount;
    System.out.println("Withdrawal successful. Amount: ₹" + amount);
}

public void displayBalance() {
    System.out.println("Current Balance: ₹" + balance);
}
}

public class BankSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        BankAccount account = new BankAccount("Akhil", 1000.0); // Initial balance ₹1000

        System.out.println("Welcome to the Bank Withdrawal System");
        account.displayBalance();
    }
}
```

```
try {  
    System.out.print("Enter amount to withdraw: ₹");  
    double amount = scanner.nextDouble();  
    account.withdraw(amount);  
} catch (InsufficientBalanceException | InvalidAmountException e) {  
    System.out.println(e.getMessage());  
}  
  
account.displayBalance();  
}
```

Output-1:

Welcome to the Bank Withdrawal System

Current Balance: ₹1000.0

Enter amount to withdraw: ₹1200

Withdrawal failed. Insufficient balance. Current balance: ₹1000.0

Current Balance: ₹1000.0

Output-2:

Welcome to the Bank Withdrawal System

Current Balance: ₹1000.0

Enter amount to withdraw: ₹500

Withdrawal successful. Amount: ₹500.0

Current Balance: ₹500.0